



PYRAX Network

Plain-English Whitepaper

Version 4.0

Publication date: 6 September 2026

<https://pyraxnetwork.org>

1 Executive summary and status

PYRAX is a high-throughput Layer-1: a base network for transferring value and supporting applications. Its design combines a web-shaped ledger, three mining and staking streams, private transfers, and higher layers for programs and transaction batches. NEURAX adds a compute marketplace; Crucible and NOVA support mining and operations.

PYRAX combines working protocol components with simulation environments, unfinished services and audit dependencies. **A working component is not evidence of production readiness.** The stated target of at least 500,000 sustained transactions per second remains an engineering target, not a demonstrated benchmark result.

Status label	Meaning
Implemented	Working code for DAG ordering, consensus signatures, the privacy proof core, execution engines and NOVA. Independent review remains pending.
Simulated / sandbox	Seed uses simplified instant mode. Broader simulation also runs on real machines.
In testing	Forge alpha and Rise testnet; testing covers protocol and proof components. Validation does not establish production readiness.
Audit-gated	PYRAX One launch and mainnet privacy circuits require outside review and a human deployment decision.
Planned / unfinished	Layer-3 coordinator, continuous cover traffic, Crucible's full build, advanced NEURAX features and vesting enforcement.

Contents

1	Summary and status	2	8	NEURAX, Crucible, NOVA	9
2	Design and architecture	3	9	Supply and allocation	10
3	DAG and GhostDAG	4	10	Fees and incentives	11
4	TriStream and finality	5	11	Governance and networks	12
5	Shielded transactions	6	12	Security and audit gates	13
6	Execution and scale	7	13	Glossary and conclusion	14
7	Networking and applications	8			

2 Design goals and architecture

PYRAX is written in Rust and separates its software into modules for ordering, cryptography, privacy, networking, storage, execution and compute services. This makes the parts easier to review separately, although their interactions still need testing.

Core property	Design intent and boundary
Concurrent blocks	A directed acyclic graph (DAG) lets blocks refer to several earlier blocks. GhostDAG establishes their order.
Default shielding	Private transfers are the default path; public transfers are an explicit choice. Mainnet privacy remains audit-gated.
Distributed participation	Multiple discovery methods and hardware classes aim to reduce reliance on one operator. Decentralization is not a proven outcome.
Metadata resistance	An onion mixnet is intended to obscure traffic patterns. Its continuous cover-traffic generator is unfinished.
Open protocol	Core rules are open source under Apache-2.0; applications and several services are proprietary.

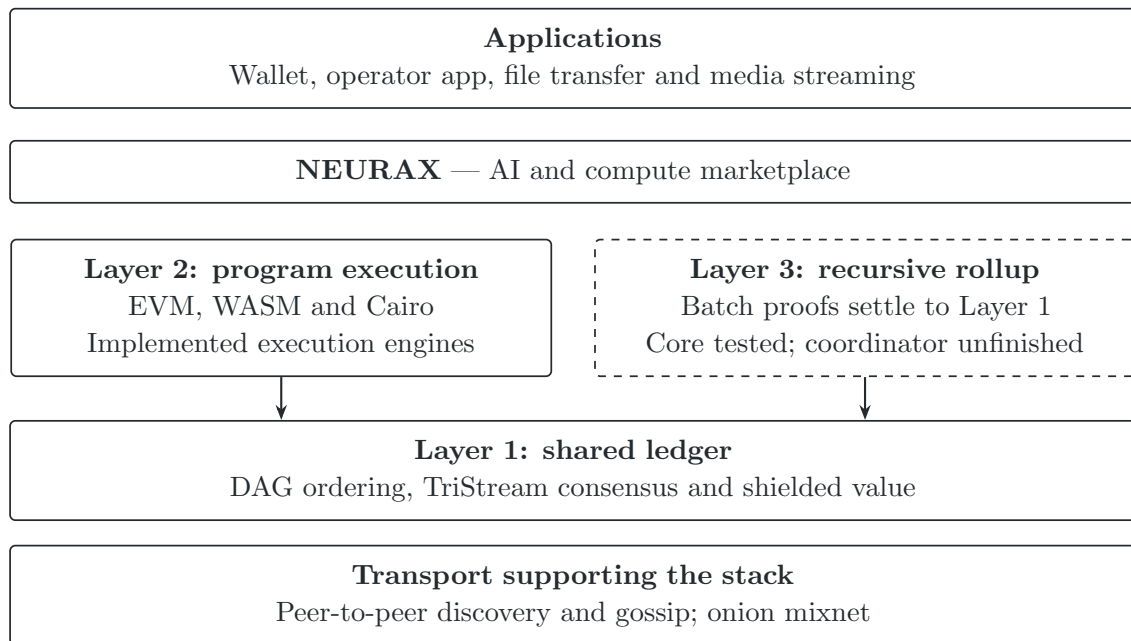


Figure 1. PYRAX’s layered architecture. Arrows show anchoring or proof settlement to Layer 1, not every data connection. Dashed outline marks the unfinished Layer-3 service.

Layer 1 records value and consensus. Layer 2 runs programs. Layer 3 is intended to batch transactions and settle compact proofs to Layer 1; it is not yet switched on as a complete service. The transport systems support communication across the design.

3 DAG and GhostDAG ordering

A block groups transactions. In a simple chain, blocks follow one another along a single history. In a **DAG**, a new block can refer to several earlier blocks. This allows concurrent work to enter the same ledger rather than forcing every block into a single line at arrival.

The graph alone does not give every transaction a unique position. PYRAX uses **GhostDAG** to derive a common block order. All three TriStream streams feed this shared structure; they are not three separate ledgers.

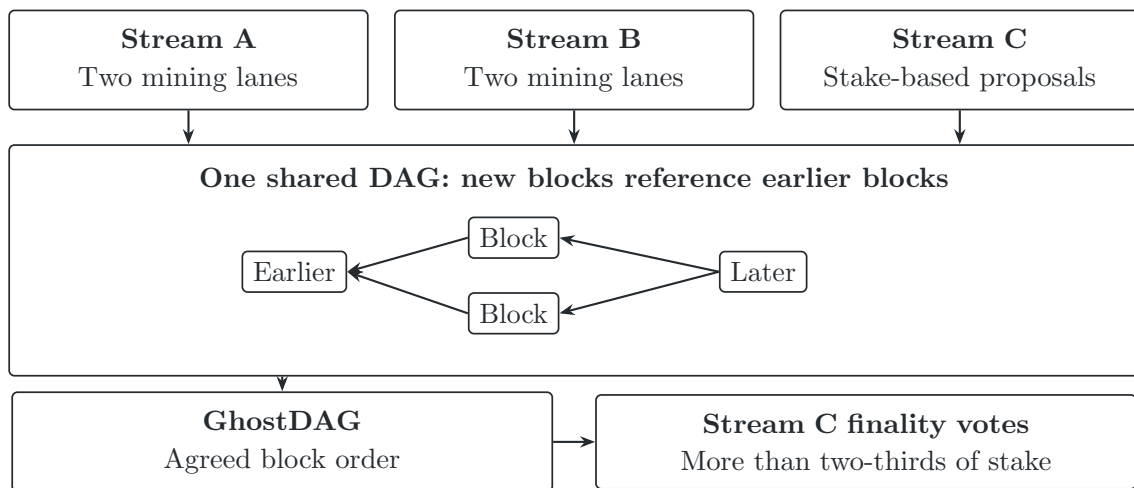


Figure 2. Three streams feed one DAG. Within the graph, arrows point from newer blocks to earlier references; the remaining arrows show processing flow. Ordering and stake-based finality are distinct steps.

What “blue” and “red” mean

GhostDAG classifies a well-connected “blue” group and a less-connected “red” group for ordering. These classifications do not directly prove that a participant is honest or malicious. The ordering rule selects the tip with the most blue work behind it.

This design aims to incorporate concurrent honest blocks while limiting the influence of withheld work. It should not be read as a guarantee that withholding never pays, that every concurrent block increases useful capacity, or that one stream can never affect another’s ordering.

4 TriStream and finality

TriStream provides three ways to produce blocks, across five lanes. Proof-of-work miners perform computation; proof-of-stake validators lock tokens and participate in proposals and voting. Different hardware requirements are intended to broaden participation.

Stream	Mechanism	Intended participants
A	BLAKE3 and double-SHA-256	Specialized mining-chip (ASIC) operators
B	kHeavyHash and Argon2id	Graphics-card (GPU) and processor (CPU) operators
C	Proof-of-stake proposals and votes	Validators with locked PYRX

Stream B uses `kHeavyHash` and `Argon2id`. The node handles Stream-A mining and Stream-C proposals, with external miners serving heavier lanes. The same GPUs and CPUs are intended to support NEURAX jobs when not mining.

Ordering is not the same as finality

Mining gives probabilistic confirmation: confidence grows as more work follows a block. Stream C adds a finality rule. Validators stake at least **32 PYRX**; votes representing **more than two-thirds of total staked weight** finalize a block under the stated protocol.

The votes use BLS signatures, which combine many signatures into one compact signature. The signature implementation includes tests and protection against rogue-key attacks, in which a maliciously chosen key can undermine signature aggregation.

Double-signing is slashable: a validator loses **5% of its stake**, with **10% of that penalty** paid to the reporter. Withdrawal takes about **seven days**. These penalties are protocol parameters, not evidence that all attacks are prevented.

The security claim needs a formal boundary

The security design aims to require control of Stream-A mining, Stream-B mining and a supermajority of stake to rewrite history. That threshold requires formal review; it is not proof that a “51% attack” is impossible. Concentrated hardware, coordinated stake, software faults and network disruption remain relevant risks.

5 Privacy and shielded transactions

A shielded transfer is designed to hide its sender, recipient and amount. PYRAX’s private ledger uses **notes**: private records of value. A public receipt, or **commitment**, records a note without exposing its contents. Spending publishes a unique **nullifier**, a one-time marker used to detect a repeated spend.

A **zero-knowledge proof** demonstrates that a transfer follows the rules without revealing its private inputs. The proof checks that notes exist, markers are correctly derived, value balances and new notes are well formed. The network also checks that spent markers have not already been used.

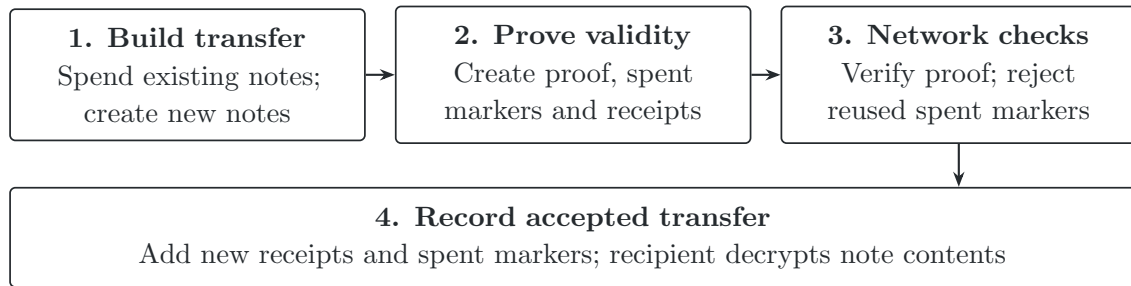


Figure 3. A shielded transfer creates a proof and public bookkeeping, not public note contents. Mainnet use of this lifecycle remains audit-gated.

What remains visible

The transaction exposes its network identifier, a reference to the note tree, spent markers, new commitments, encrypted note contents, the proof and a net-value field. This field is zero for a fully private transfer. A small flat shielded-transaction fee is burned.

Shielding moves public value into the private pool. **Deshielding** moves it back out. A fully private transfer stays within the pool. These paths should not be treated as having identical exposure: the public side of a deposit or withdrawal is not itself shielded.

Implementation and limits

The proof system works without a trusted setup, a special initial ceremony that some proof systems require. Its circuits are pinned: nodes check that the proof rules match expected versions. Neither property removes the need to review the circuits and their integration.

Shielding does not by itself hide internet traffic patterns, protect a compromised wallet or make all transfers indistinguishable. Duplicate-marker checks reject repeat spends under the protocol’s assumptions; “double-spending is impossible” is too broad a claim.

6 Execution and scalability

A smart contract is a program whose results are checked by the network. PYRAX places richer program execution above Layer 1. Layer 2 provides three virtual machines: software environments that execute these programs and can call one another.

Environment	Purpose	Intended audience
EVM	Ethereum-style contracts in Solidity or Vyper	Ethereum application developers
WASM	Sandboxed contracts in several languages	General-purpose developers
Cairo	Proof-friendly contracts	Developers using provable computation

MetaMask and Foundry compatibility requires confirmation through a test matrix. PYRAX supports six transaction types: shielded, transparent, Ethereum-style, compute-escrow, staking and governance. Compute escrow locks payment before a job is settled.

Layer 3: working proof core, unfinished service

A rollup groups transactions outside the base ledger and settles their results back to it. PYRAX's proposed coordinator collects batches, a prover checks them mathematically, and recursive folding combines proofs into a compact proof for Layer 1 to verify.

The validity proof, folding and on-chain checker are implemented and tested. **The coordinator is unfinished, so the complete Layer-3 service is not switched on.** Working proof components do not demonstrate end-to-end throughput.

The 500,000 TPS target

The target is **at least 500,000 sustained transactions per second across the network** for simple value transfers. It is not a measured base-layer result. Contract calls and private transfers need separate measurements.

The scaling approach combines concurrent blocks, simultaneous execution of non-conflicting transactions, upper-layer batching, proof folding, GPU-assisted proving and data-availability sampling. Sampling lets light devices check selected pieces instead of downloading all data. PYRAX defines archival, full, light and mobile node classes; hardware requirements for each class still need to be specified.

These techniques shift costs among block producers, provers, storage providers and users. Reaching the target without increasing participation requirements remains to be demonstrated.

7 Networking and user applications

Joining without one required starter server

PYRAX uses bootstrapless peer discovery: a new node can discover peers through local discovery, a distributed directory crawl, peer referrals and a community-maintained signed seed list. The intent is to avoid a mandatory company-operated entry point, not to eliminate all trust in discovery inputs.

Blocks and shielded transactions spread through peer-to-peer gossip. Network safeguards include spam controls, connection limits, misbehavior scoring and a cap of **eight peers per network neighborhood** to reduce eclipse risk—an attack that isolates a node behind attacker-controlled peers.

Shielding content is not hiding traffic

An onion mixnet wraps messages in layers of encryption and sends them through relays. Each relay removes a layer to forward the message. Fixed-size packets are intended to reduce clues from packet length; cover traffic adds decoy messages to obscure when users are active.

Core onion routing is implemented; continuous cover traffic is unfinished. Encryption and fixed packet sizes do not establish that an internet provider or a traffic observer cannot correlate communications. The aim is resistance to traffic analysis, not guaranteed anonymity.

File transfer and media streaming

Resumable file transfers use encrypted chunks, content fingerprints and access-token-gated keys. A fingerprint alone does not grant access to the plaintext. Media streaming uses short encrypted segments distributed through a peer swarm. Both services are designed to use the mixnet.

Rate limits and access-token allow/deny lists are the stated abuse controls. Payments to relay and storage providers are planned marketplace features.

Apps, wallet and update risks

Inferno is the public desktop operator app for running a full node, mining, staking and contributing compute. An internal seeding app and a command-line tool that embeds the node are also available. Apps connect to a per-node web portal through a self-hosted secure tunnel.

Desktop key protection uses password-based encryption, the operating-system keychain and an additional two-factor code. The private-spending key stays in the app rather than being sent to the node. The updater uses secure transport and operating-system code signing but lacks an extra application-level download-signature check. Command-line key protection is weaker than the desktop wallet's.

8 NEURAX, Crucible and NOVA

NEURAX: paid compute with layered verification

NEURAX aims to put mining GPUs and CPUs to work on AI inference and other jobs. Payment is locked on-chain, workers run jobs in a sandbox, results are checked, and settlement pays a verified cryptographic key rather than a self-declared identity.

NEURAX uses four verification methods: repeated execution on identical hardware and software; fraud challenges within a time window; interactive disputes that narrow down a disagreement; and attested trusted execution environments (TEEs). A TEE is hardware intended to isolate a computation and report what ran. These methods have different trust assumptions; none proves all large AI computations correct.

Trust tier	Design and limitation
Open	Consumer GPUs can participate; result checking still matters.
Secure	Open models split across consumer GPUs. A deterrent to disclosure, not a cryptographic privacy guarantee.
Trusted	Proprietary models use sealed datacenter hardware. Advanced attestation remains unfinished.

NEURAX is local-first, with an RTX 3060 as the stated baseline target. Routing chooses local or network execution. Large models run slowly over home internet, and consumer AI-video output remains limited. Much of NEURAX is built and tested, while hardware attestation, the public gateway, coding copilot and high-end video remain ahead.

Crucible: planned zero-developer-fee mining

Crucible is a first-party miner-manager designed to connect to the operator's local node and support Stream-A ASICs and Stream-B GPUs/CPUs. Its stated developer fee is zero. That does not remove electricity, equipment or other operating costs, nor establish profitability. Crucible is designed and scaffolded; its full build is deferred until the core mainnet program is complete.

NOVA: advisory-first operations

NOVA is implemented and operational. It monitors services, gathers evidence and can propose or apply restricted repairs through an AI coding assistant. Autonomy is off by default. Keys, wallets and consensus rules are excluded; high-risk actions require a person, and chain-service restarts require a **15-minute warning**.

The stated workflow tests changes before shipping, rolls back failures, runs its AI locally and keeps an audit log. These controls reduce intended scope; they do not guarantee safe repairs or an unchangeable record.

9 Token supply and allocation

The token is **PYRAX**, ticker **PYRX**. One PYRX has 18 decimal places; its base unit is called **ash**. The protocol specifies a maximum of **50 billion PYRX**: 37.5 billion at genesis and up to 12.5 billion in block rewards. Seed is explicitly exempt from the supply cap; see Section 11.

Table 1. Token allocation totals 50 billion PYRX and 100%. Issuance and enforcement require independent verification.

Pool	Billion PYRX	Share	Stated use
Genesis distribution	25.0	50%	Public sale and bonus
Ecosystem / liquidity	5.0	10%	Grants, integrations, liquidity
AI-Compute	4.0	8%	Compute-provider payments
Team / advisors	2.5	5%	Team allocation
DAO treasury	1.0	2%	Community reserve
Block rewards	12.5	25%	Mining / stream issuance
Total	50.0	100%	

The operations treasury starts at zero and receives fee revenue. It is separate from the one-billion-PYRX DAO allocation.

Launch terms, not evidence of a completed sale

The genesis price is set at **US\$0.0025 per PYRX**. Offering 20 billion tokens at that price would raise US\$50 million *if fully sold*. A further five billion tokens form a 25% bonus described as network-access or compute credits. These figures do not establish sales, market value or returns. The implied fully diluted valuation is US\$125 million at that stated price.

Intended vesting; enforcement is a later phase

Allocation	Intended release schedule
Buyers and bonus	Fully liquid at launch
Team / advisors	One-year cliff, then release over 36 months
Ecosystem / liquidity	40% at launch; remainder over 24 months
AI-Compute	Streamed over 48 months
DAO treasury	10% at launch; remainder by community vote

The listed immediate releases total 27.1 billion PYRX before any initial AI-pool release. Vesting enforcement and DAO treasury voting remain later-phase work.

10 Fees, emissions and incentives

Issuance and validator rewards

The initial block reward is **300 PYRX**, halving every **21 million blocks**, until the 12.5-billion reward cap is reached. The estimated schedule allows four years per halving and roughly 26 years of issuance, depending on actual block timing. The full reward goes to the block producer; roughly equal block production across streams is intended to spread issuance roughly evenly.

Stream-C validators receive block rewards, tips on their blocks and staking rewards. A separate staking-reward schedule still needs to be specified. Once issuance ends, producer income depends on fees; the listed split directs producer fee income through tips, not the base fee. At exactly four years per halving, the first eight years issue 9.45 billion PYRX, or 75.6% of the reward cap, and the cap is reached near year 28. The approximate 26-year estimate therefore depends on different block timing.

The stated fee split

PYRAX uses a base fee plus an optional tip, described as an EIP-1559-style model. The following split cannot be changed through on-chain governance. Shielded transfers also pay a small flat fee that is burned; its amount still needs to be specified.

Table 2. Protocol distribution of the base fee and optional tip.

Component	Burned	Producer	Operations	DAO
Base fee	25%	0%	50%	25%
Tip	0%	70%	20%	10%

Compute pricing and subsidy risk

A Compute Unit (CU) is defined as one hour of an RTX 4090 reference GPU. The stated rate is **8 PYRX per CU**, with quality adjustments of **0.8–1.2 times** and buyer priority payments up to **2 times**. The planned AI-pool payout rate is approximately **70 million PYRX per month**.

Job fees and the treasury's share of network fees are intended to replace subsidy funding over time. Any remaining pool would return to the DAO once revenue covers payouts. That transition is a design assumption, not demonstrated demand or revenue.

At the stated genesis price, 8 PYRX equals US\$0.02 per reference GPU-hour. This is not a profitability forecast: providers bear costs, and token appreciation cannot be assumed. At 70 million PYRX per month, payouts total 3.36 billion over 48 months. The four-billion pool and its release schedule are distinct from this baseline payout calculation.

11 Governance and four networks

Anyone may submit a proposal with a **1,000-PYRX refundable deposit**. Validator votes are weighted by stake. Approval requires at least **one-third participation**, more than **two-thirds approval**, and a **one-day delay** before an approved change takes effect.

Voting can change only the block-size limit, minimum validator stake and withdrawal wait. The governance mechanism excludes fee splits, the supply cap, halving schedule and governance rules from this mechanism. **DAO treasury spending by vote is planned**, not an available governance function established here.

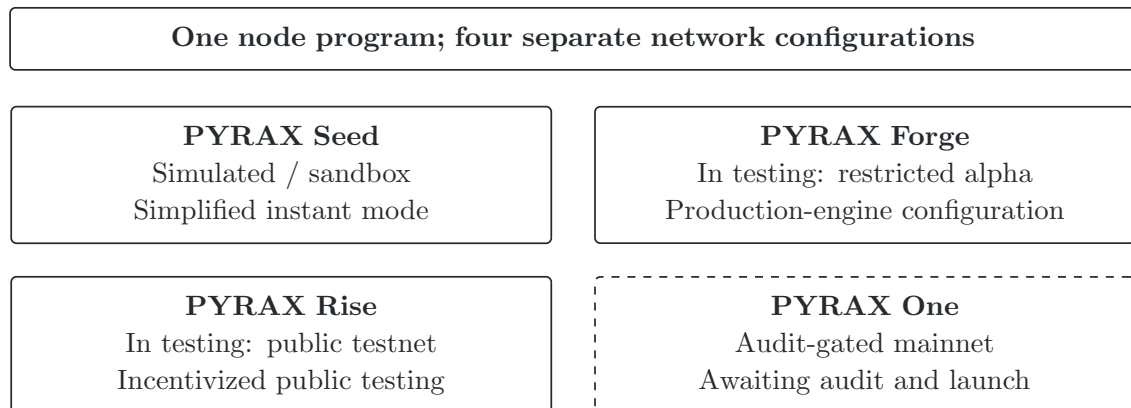


Figure 4. Four separate environments, not a migration pipeline. Solid boxes show sandbox or testing roles; the dashed box marks the audit-gated mainnet. Deployed modes still require confirmation.

Network	Chain ID	Role and important boundary
PYRAX Seed	881109	Permanent play-money sandbox; only environment without the hard supply cap.
PYRAX Forge	710823	Restricted alpha with a small initial validator set and a production-engine configuration.
PYRAX Rise	104928	Public incentivized testnet for testing the system.
PYRAX One	563821	Mainnet after external review; no faucet or sandbox shortcuts.

Production configurations reject Seed's instant mode. That guard is not proof of mainnet launch or audit completion. Forge's eligibility and access rules require clarification.

12 Security limits and audit gates

PYRAX maintains tests for protocol invariants: supply limits, value conservation, finality and prevention of invalid private spends. A threat model sets out relevant attackers and safeguards. Both require independent assessment; tests do not substitute for external review.

Area	Assumption, limitation or dependency
Consensus	GhostDAG ordering, stream weighting and stake voting must work together. The claimed multi-resource attack threshold lacks supporting analysis here.
Privacy	Correct circuits, wallet handling and verification are essential. No trusted setup does not mean no cryptographic risk; mainnet circuits require audits.
Network metadata	Relay behavior and traffic patterns matter. Continuous cover traffic is incomplete; anonymity is not guaranteed.
Scaling / compute	Layer-3 coordination and advanced NEURAX attestation are unfinished. Large AI jobs are not universally verified by cryptographic proofs.
Apps / operations	Update verification and command-line key protection have known gaps. NOVA's controls need review; passing tests cannot guarantee a safe change.
Economics / control	Vesting enforcement and DAO spending are later work. Token concentration, costs, demand and fee income affect participation incentives.

Conditions before PYRAX One launches

Mainnet preparation and review require the following steps, without fixed completion dates:

1. Fuzzing: automated malformed and unexpected inputs for consensus, ordering, data formats and circuits.
2. Dedicated external cryptography and circuit audits, plus review of consensus and privacy design. The audit scope also includes a bridge.
3. An incentivized Rise testnet and a bug bounty.
4. A deliberate human deployment decision. No privacy circuit is to reach mainnet without independent review.

Open-core does not mean every component is open

Apache-2.0 covers the node, consensus, privacy circuits, command-line tools, developer kit and contract interfaces. Apps, the wallet, NEURAX and hosted services are proprietary. Public protocol code enables inspection; it does not itself establish security, distributed control or freedom from operational dependencies.

13 Glossary and conclusion

Term	Plain-English meaning in this document
Layers 1, 2 and 3	Base ledger; program execution; proposed recursive batching and settlement.
DAG	Directed acyclic graph: blocks link to earlier blocks without forming loops.
GhostDAG	The algorithm used to derive an agreed order from the DAG.
TriStream	PYRAX's three block-production streams: two mining streams and one stake-based stream.
Finality	The protocol's rule for treating a block as settled, subject to its security assumptions.
Staking / slashing	Locking tokens to validate; a penalty that removes some stake for specified misbehavior.
Shielded transaction	A transfer designed to conceal sender, recipient and amount while proving validity.
Note / commitment	A private value record; its public cryptographic receipt.
Nullifier	A one-time spent marker used to reject repeated spending of a note.
Zero-knowledge proof	Evidence that a statement is true without revealing its private inputs.
Rollup / proof folding	Batching transactions for settlement; combining proofs into a compact proof.
Mixnet / cover traffic	Relays intended to obscure message paths; decoy messages intended to hide activity patterns.
TEE / attestation	Hardware-isolated execution; evidence about the hardware and software running a job.
DAO / escrow	A community-governed treasury structure; payment locked until settlement conditions are met.

Conclusion

PYRAX combines a shared DAG, TriStream, default shielding, multiple execution environments and a compute marketplace. Core components are implemented, while several services remain unfinished. Mainnet readiness depends on outside review and a human deployment decision.

Implementation is not the same as demonstrated performance or independent assurance. The throughput target, sustainable provider revenue and production readiness remain to be established through testing, operation and external review.